

MongoDB



Luigi Santangelo
luigi.santangelo@unipv.it

16 december 2025

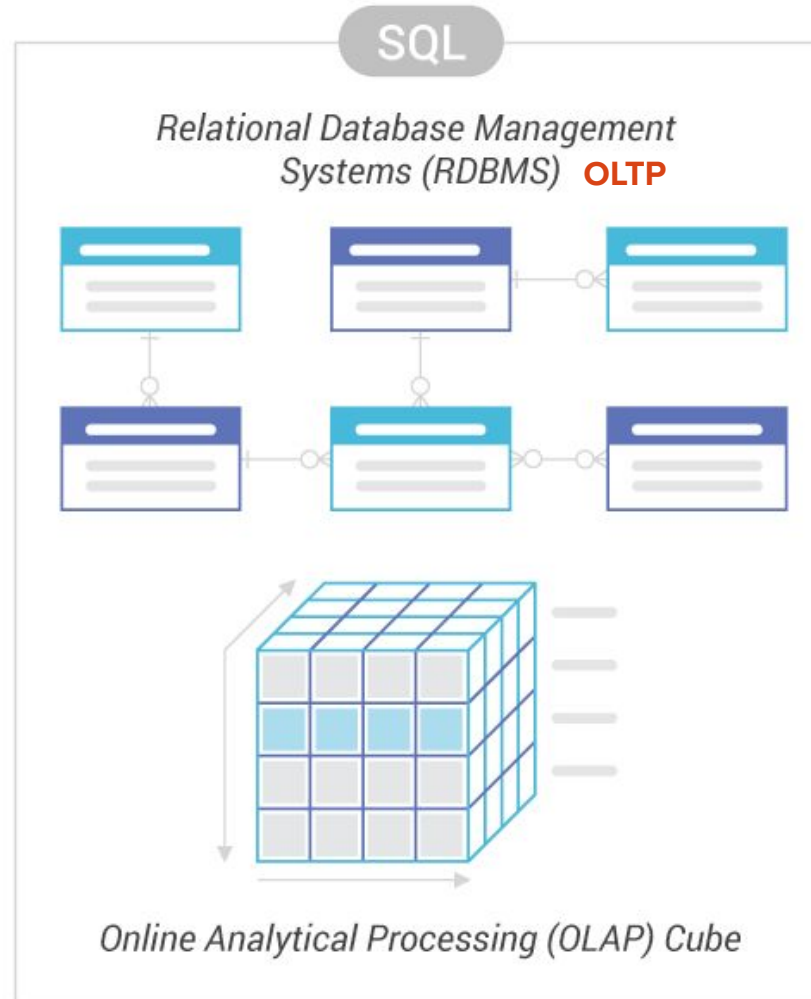
Overview

- MongoDB: document-oriented database

What we do:

- Relation between SQL and NoSQL systems
- Basic features of MongoDB
- Practical Session: storing and retrieving documents

The traditional databases



Many SQL Databases

ORACLE



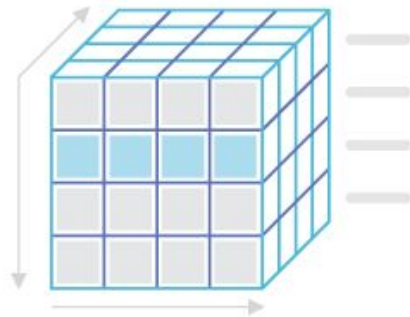
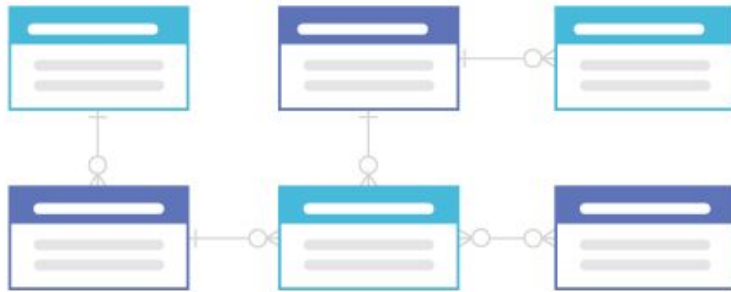
PostgreSQL



Two types of databases

SQL

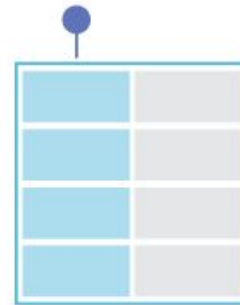
Relational Database Management Systems (RDBMS)



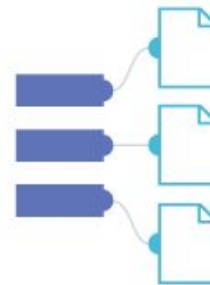
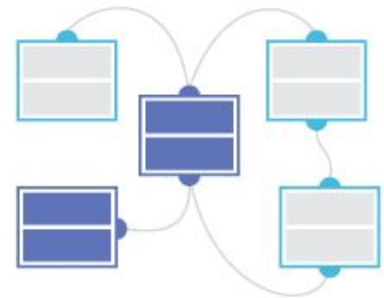
Online Analytical Processing (OLAP) Cube

NoSQL

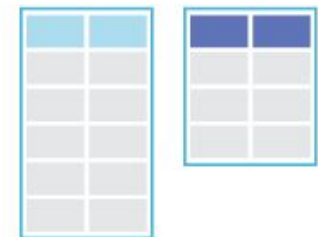
Key-Value



Graph



Document



Column store

Taxonomy of NoSQL

Document Database	Graph Databases
    	 
Wide Column Stores	Key-Value Databases
   	     

Which database should we choose?

Choosing the right database is not easy, as all of them provide different and interesting features.

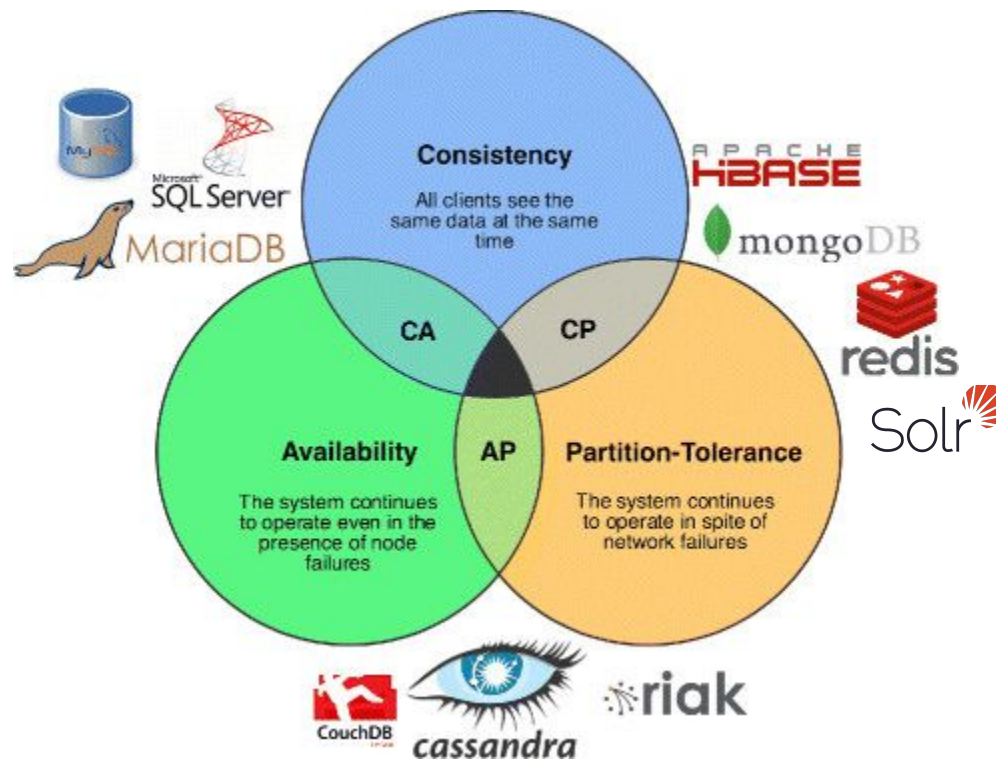
What are we looking for in a database?

- Consistency: all clients see the same data at the same time
- Availability: the system continues operating even in the presence of a node failure
- Partition-Tolerance: the system continues operating despite of network failure

The CAP Theorem

Sadly, according to the CAP Theorem, it is **impossible for a distributed data store** to simultaneously provide more than two of the three features.

Depending on what we are **willing to get rid of**, we should decide our best database:



What is MongoDB

- Document-Oriented
- Non-relational database
- Classified as NoSQL (no support to SQL)
- Each document is written in JSON
- The document structure can varies document by document
- Document structure can be modified any time
- It provides high availability
- it is able to scale over 100 nodes
- Speed: it supports more than 100.000 read and write operations in a second
- It is able to store over one billion of data
- It's open source
- It's multiplatform
- There are API for many programming languages

Who is using MongoDB?

Google

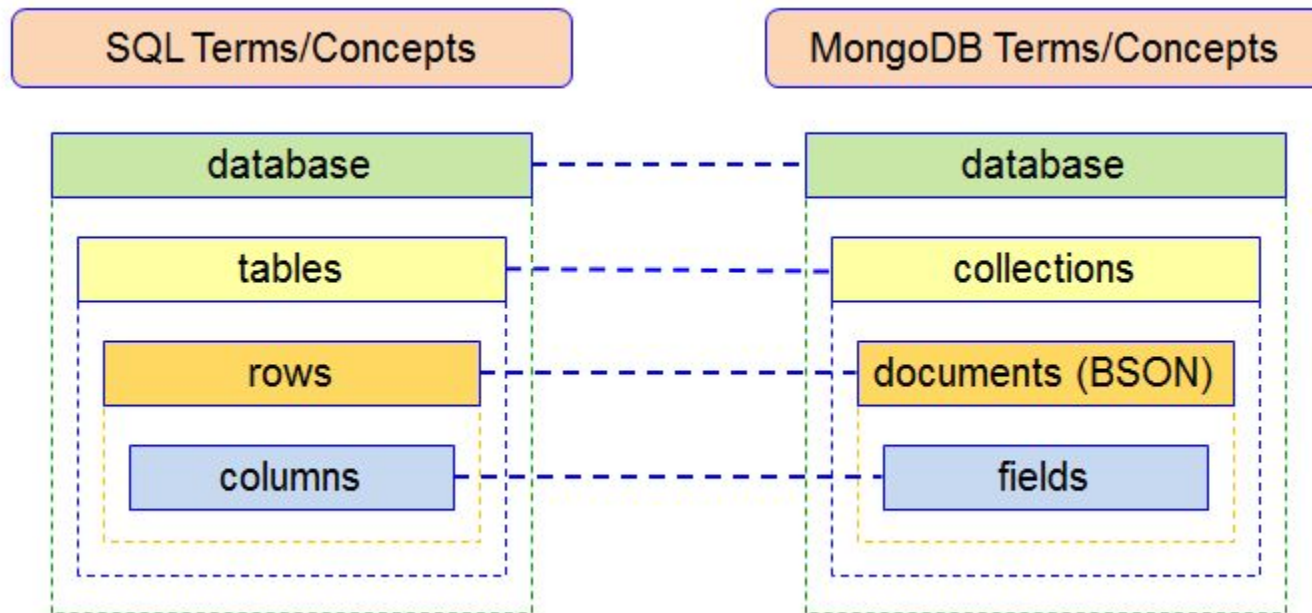
facebook

PayPal

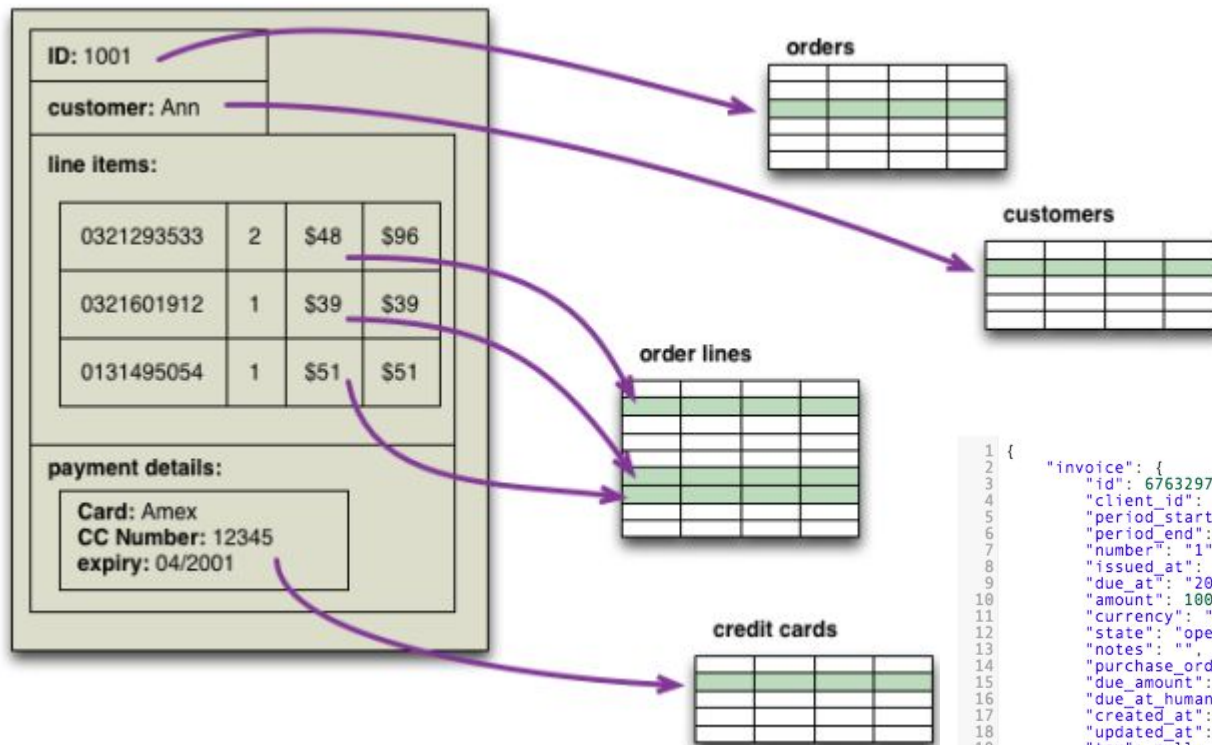


Terminology

Although terminology on MongoDB is a bit different than Relational Databases, it is possible to make a mapping between both systems



A document



A document:

- max 16 MB
- BSON format (consisting of field-value pairs)

```
1 {
2   "invoice": {
3     "id": 6763297,
4     "client_id": 1929151,
5     "period_start": null,
6     "period_end": null,
7     "number": "1",
8     "issued_at": "2015-04-22",
9     "due_at": "2015-04-22",
10    "amount": 100,
11    "currency": "United States Dollar - USD",
12    "state": "open",
13    "notes": "",
14    "purchase_order": "",
15    "due_amount": 100,
16    "due_at_human_format": "upon receipt",
17    "created_at": "2015-04-21T18:41:58Z",
18    "updated_at": "2015-04-21T18:42:02Z",
19    "tax": null,
20    "tax_amount": 0,
21    "subject": "",
22    "recurring_invoice_id": null,
23    "tax2": null,
24    "tax2_amount": 0,
25    "client_key": "43d9342a017e262c33a395ef3b9dca294f736792",
26    "estimate_id": null,
27    "discount": null,
28    "discount_amount": 0,
29    "retainer_id": null,
30    "created_by_id": 508343,
31    "csv_line_items":
32    "kind,description,quantity,unit_price,amount,taxed,taxed2,project_id\nProduct,A
33  }
```

The big difference

Main differences between relational databases and document oriented DB

- RDBMS defines data structures at **table level** (*all records need to have the same structure*)
 - storing new object with new data requires extending the structure
- Document Oriented DB defines data structure at **document level** (*all documents can have different structures*)

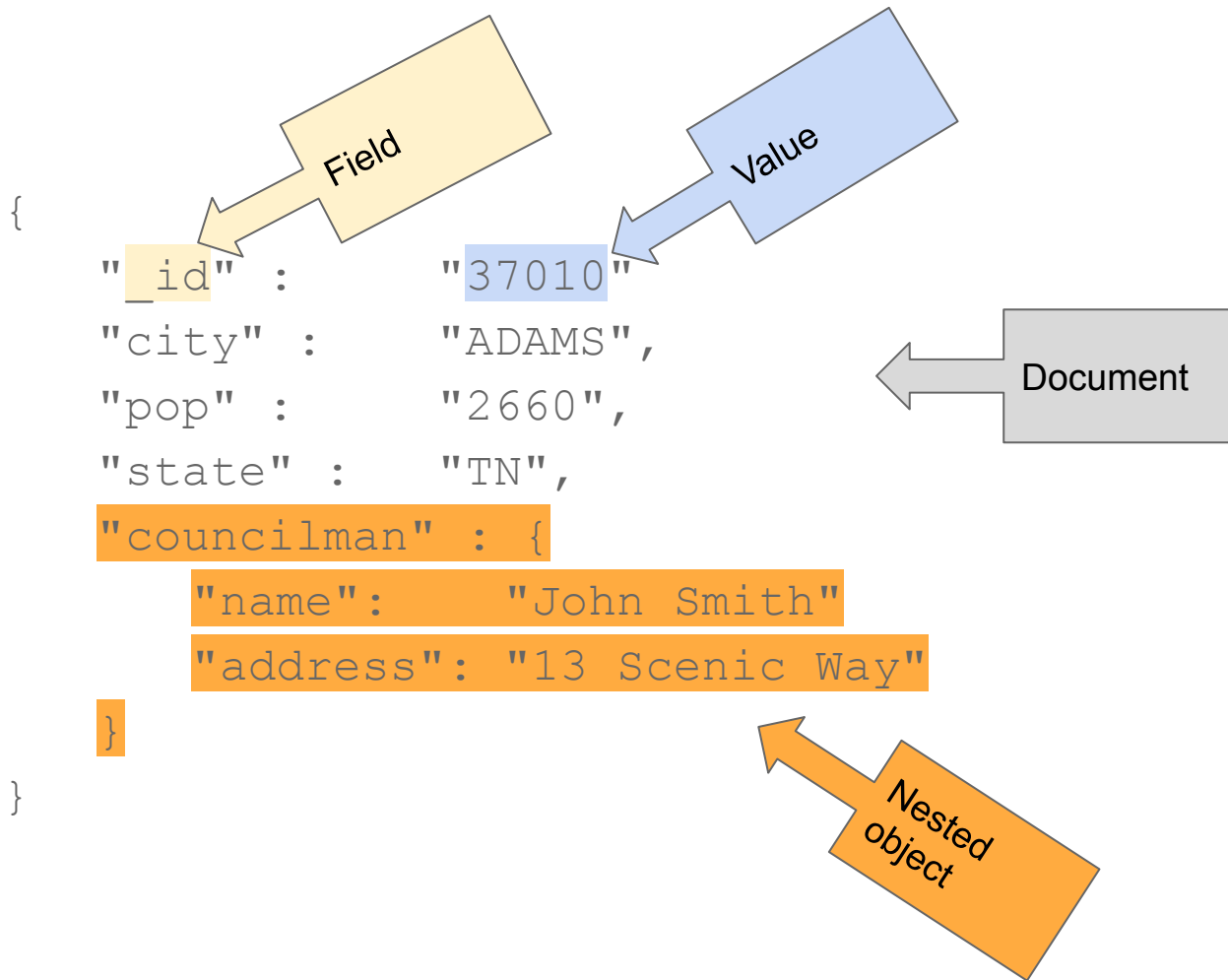
A document is **schema-less**. Different documents belonging to the same collection can have a different field set

JSON and BSON

- JSON
 - short for JavaScript Object Notation
 - Easy for humans to write/read
 - Easy for computers to parse/generate
 - Object can **be nested**
- BSON
 - Binary JSON

```
{
  id: "ttl231",
  title: "Pride and Prejudice"
  year: 2093,
  director: "Michael Bay",
  genres: [
    "Horror",
    "Comedy"
  ],
  stars: [
    {
      name: "Kiera Knightley",
      id: 9863
    },
    {
      name: "Danny DeVito",
      id: 2031
    }
  ]
}
```

JSON Example



Data Types

Type	Number	Alias	Notes
Double	1	"double"	
String	2	"string"	
Object	3	"object"	
Array	4	"array"	
Binary Data	5	"binData"	
Undefined	6	"undefined"	Deprecated
ObjectId	7	"objectId"	
Boolean	8	"bool"	
Date	9	"date"	
Null	10	"null"	
Regular Expression	11	"regex"	
DBPointer	12	"dbPointer"	Deprecated
JavaScript	13	"javascript"	
Symbol	14	"symbol"	Deprecated
Javascript (with scope)	15	"javascriptWithScope"	
32-bit integer	16	"int"	
Timestamp	17	"timestamp"	
64-bit Integer	18	"long"	
Decimal128	19	"decimal"	New in Version 3.4
Min Key	-1	"minKey"	
Max Key	127	"maxKey"	

The `_id` field

```
{
  "_id" :      "37010"
  "city" :     "ADAMS",
  "pop" :      "2660",
  "state" :    "TN",
  "councilman" : {
    "name":     "John Smith"
    "address":  "13 Scenic Way"
  }
}
```

- By default, each document contains an `_id` field.
- The field is used as Primary Key
- Its value is unique, immutable, and may be any non-array types

Installation (Server)

- browse on
<https://www.mongodb.com/try/download/community-kubernetes-operator>
- choose the preferred version, the operating system and the package
 - 8.0.4
 - RedHat / Centos 8.0 x64
 - TGZ
- Download

otherwise

- https://fastdl.mongodb.org/linux/mongodb-linux-x86_64-rhel8-8.0.4.tgz
- `tar -xzvf mongodb-linux-x86_64-rhel8-8.0.4.tgz`
- `mv mongodb-linux-x86_64-rhel8-8.0.4.tgz /usr/local/mongodb-8.0.4`
- `cd /usr/local/mongodb-8.0.4`

Configuration and Starting

- `mkdir /etc/mongodb`
- `mkdir /etc/mongodb/data`
- `cd bin`
- `./mongod --dbpath=/etc/mongodb/data/`



Path to install the database on

Server is started on port 27017

Installation (Client)

- <https://www.mongodb.com/try/download/shell>
- Select your preferred version:
 - 2.3.6
 - Linux x86
 - tgz
- or use the link:
<https://downloads.mongodb.com/compass/mongosh-2.3.6-linux-x64.tgz>
- tar -xvf [mongosh-2.3.6-linux-x64.tgz](#)
- cd bin
- ./mongosh
 - Connecting to: mongodb://127.0.0.1:27017/

How to use the client?

- mongosh

Some commands

> show databases

> db.help() shows all methods

> db.stats() shows information about the database

> db.version() MongoDB server version

> db.getName() name of the active database

> use learn set learn as predefined. If learn is missing, it will be created when the first collection is created

> db.getCollectionNames() shows all collections. If no collections are available, symbol [] is shown

Creating a document

As the collection is **schema-less**, each collection is created **without** defining its structure. The structure indeed is related to the document, not to the collection. Because of this, it doesn't exist any command to create collection, but the collection will be created as soon as the first document is created.

- > use learn
- > db.stats()
- > db.getCollectionNames()

New Documents

```
> db.unicorns.insert(  
  {name: 'Horny',  
    dob: new Date(1992,2,13,7,47),  
    loves: ['carrot', 'papaya'],  
    weight: 600,  
    gender: 'm',  
    vampires: 63}  
);
```

Each document is identified by an id (which is usually added by default)

Each field has its own data type (String, date, Integer, ...)

```
> db.unicorns.insert(  
  {name: 'Goofy',  
    dob: new Date(1994,7,21,2,22)}  
);
```

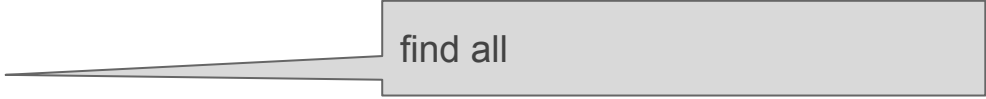
Documents can have different structures

Inserting unicorns

- `db.unicorns.insert({name: 'Horny', dob: new Date(1992,2,13,7,47), loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});`
- `db.unicorns.insert({name: 'Aurora', dob: new Date(1991, 0, 24, 13, 0), loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});`
- `db.unicorns.insert({name: 'Unicrom', dob: new Date(1973, 1, 9, 22, 10), loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});`
- `db.unicorns.insert({name: 'Roooooodles', dob: new Date(1979, 7, 18, 18, 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});`
- `db.unicorns.insert({name: 'Solnara', dob: new Date(1985, 6, 4, 2, 1), loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});`
- `db.unicorns.insert({name: 'Ayna', dob: new Date(1998, 2, 7, 8, 30), loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});`
- `db.unicorns.insert({name: 'Kenny', dob: new Date(1997, 6, 1, 10, 42), loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});`
- `db.unicorns.insert({name: 'Raleigh', dob: new Date(2005, 4, 3, 0, 57), loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});`
- `db.unicorns.insert({name: 'Leia', dob: new Date(2001, 9, 8, 14, 53), loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});`
- `db.unicorns.insert({name: 'Pilot', dob: new Date(1997, 2, 1, 5, 3), loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});`
- `db.unicorns.insert({name: 'Nimue', dob: new Date(1999, 11, 20, 16, 15), loves: ['grape', 'carrot'], weight: 540, gender: 'f'});`
- `db.unicorns.insert({name: 'Dunx', dob: new Date(1976, 6, 18, 18, 18), loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165});`

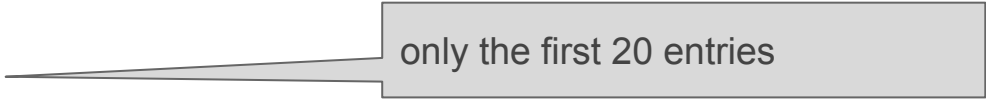
Finding documents and limiting results

> db.unicorns.find()



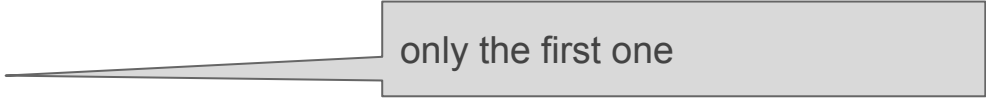
find all

> db.unicorns.find().limit(20)



only the first 20 entries

> db.unicorns.findOne()



only the first one

Selectors

They are the same as the **WHERE clause** in the Relational Database

This is the syntax:

```
{ field: value }
```

for example, let's find all unicorns having name Aurora

```
> db.unicorns.find({ "name" : "Aurora" });
```

Combining Selectors with Operators

- and
 - `db.unicorns.find({$and: [{'name': 'Aurora'}, {'weight': 450}]});`
- lower than
 - `db.unicorns.find({"weight": $lt: 700});`
- greater than
 - `db.unicorns.find({"weight": $gt: 700});`
- lower or equal to
 - `db.unicorns.find({"weight": $lte: 700});`
- greater or equal to
 - `db.unicorns.find({"weight": $gte: 700});`
- not equal
 - `db.unicorns.find({"weight": $ne: 700});`
- In
 - `db.unicorns.find({"loves": $in: ["carrot", "watermelon"]});`
- Or
 - `db.unicorns.find({$or: [{'loves': 'carrot'}, {'weight': $gt: 600}]});`

Examples

Let's find all unicorns having a weight between 400 and 700:

```
> db.unicorns.find({ $and: [ {weight: {$gt: 400}}, {weight: {$lt: 700}} ] })
```

Let's find all unicorns which like carrot and apple, or have a weight greater than 700

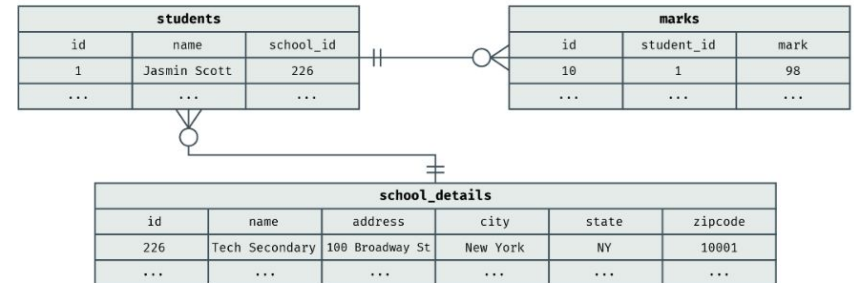
```
> db.unicorns.find({$or: [{ $and: [{"loves": "carrot"}, {"loves": "apple"}]}, {"weight": {$gt: 700}}]})
```

Let's find all unicorns which are born between 1991, the 1st january and 1997, the 31st december:

```
> db.unicorns.find( { $and: [{"dob": {$gte: new Date("1991-01-01")}}, {"dob": {$lte: new Date("1997-12-31")}}] })
```

MongoDB vs SQL

```
{
  "_id": 1,
  "student_name": "Jasmin Scott",
  "school": {
    "school_id": 226,
    "name": "Tech Secondary",
    "address": "100 Broadway St",
    "city": "New York",
    "state": "NY",
    "zipcode": "10001"
  },
  "marks": [98, 93, 95, 88, 100],
}
```



Results

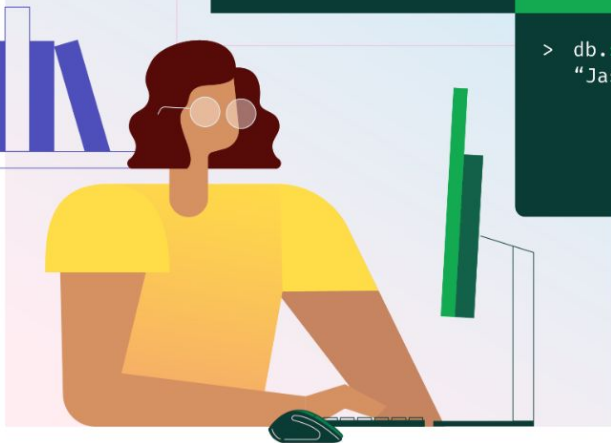
name	mark	school_name	city
Jasmin Scott	98	Tech Secondary	New York
...	...	...	...

mongo

```
> db.students.find({"student_name":
  "Jasmin Scott"})
```

sql

```
SELECT s.name, m.mark, d.name as "school name",
d.city
FROM students s
INNER JOIN marks m ON s.id = m.student_id
INNER JOIN school_details d ON s.school_id = d.id
WHERE s.name = "Jasmin Scott";
```



Regular expressions

- ^ at the beginning
- \$ at the end
- .
- * zero, one or more characters
- + one or more characters

Examples

Let's find all unicorns having a "i" as second letter in their name

```
> db.unicorns.find({"name": {$regex: "^i"}})
```

Let's find all unicorns having the name ending with "a"

```
> db.unicorns.find({"name": {$regex: ".*a$"}})
```

Delete e Drop

Selectors can also be used not just in queries but also in delete operations

Delete all unicorns

```
> db.unicorns.deleteMany({});
```

Delete all unicorns which like grapes

```
> db.unicorns.delete("loves": "grape")
```

Drop the whole unicorn collection

```
> db.unicorns.drop();
```


Update: replace e set

The update command makes an update of a single document. It needs two parameters: a selector and an update option

BE CAREFULL: this command requires that all the document is specified: For example, the following command

```
> db.unicorns.update({name: "Roooooodles"}, {weight: 590})
```

does not update the Roooooodles weight to 590, but **it replaces** the document having name Roooooodles with another document containing just the weight field.

Indeed if we make a search, we can notice that it exists a document having a different structure from the others

```
> db.unicorns.find();
```

Update: replace e set

If we wanted to modify just the weight field, we would have to use the **\$set operator**

```
> db.unicorns.update({name: "Rooooooodles"}, {$set: {weight: 590}})
```

Update: inc e push

Beyond \$set operator, there are many others available.

\$inc increments or decrements a field. For example, let's reduce the number of vampires killed by Aurora

```
> db.unicorns.update({name: "Aurora"}, {$inc: {vampires: -2}})
```

\$push instead works on arrays. For examples if Aurora loves also sweets, we can use the following command:

```
> db.unicorns.update({name:"Aurora"}, {$push: {loves: "sweets"}})
```

Other operators

For further operators, please see

<https://docs.mongodb.com/manual/reference/operator/update/>

Multiple updates

By default, the update command works **only on the first document**. If we want to update all retrieved documents, we have to use the multi parameter:

For example, put true on the vaccinated field for all unicorns

```
> db.unicorns.update({}, {$set: {vaccinated: true}}, {multi: true});
```

Sorting

All documents retrieved by find command can be ordered

Let's order in ascending mode all retrieved unicorns

```
> db.unicorns.find({}).sort({name: 1})
```

Let's order all unicorns by weight in descending mode and by age in ascending mode

```
> db.unicorns.find({}).sort({weight: -1, dob: 1});
```

Field selectors

By default, find command shows all fields for each document. It is possible to limit the fields to show, by using a second parameter in the find method, specifying all fields to be displayed

For example, let's show only name and weight of all unicorns

```
> db.unicorns.find({}, {name: 1, weight: 1});
```

Pagination

If the list of documents retrieved using find method is too long, it can be truncated using limit command

Let's show only the first two unicorns

```
> db.unicorns.find({}).limit(2)
```

Show only the second and the third most weight unicorns:

```
> db.unicorns.find({}, {name: 1, weight: 1}).sort({weight: 1}).limit(2).skip(1);
```


Counting

To count the number of documents in a collection:

```
> db.unicorns.count();
```

For example: let's count the number of unicorns having a weight greater than 500

```
> db.unicorns.find({weight: {$gt: 500}}).count();
```

Relations vs Embedding & Linking

User

- Name
- Email address

Article

- Name
- Slug
- Publish date
- Text
- Author

Comment[]

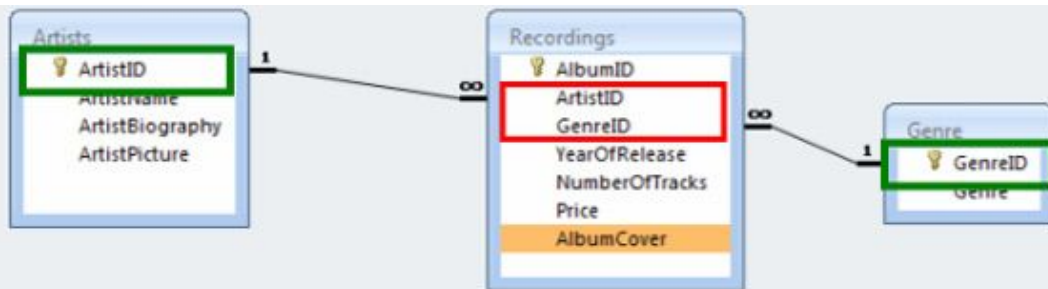
- Comment
- Date
- Author

Tag[]

- Value

Category[]

- Value



= primary key



= foreign key

Embedding

```
book = {  
  title: "MongoDB: The Definitive Guide",  
  authors: [ "Kristina Chodorow", "Mike Dirolf" ]  
  published_date: ISODate("2010-09-24"),  
  pages: 216,  
  language: "English",  
  publisher: {  
    name: "O'Reilly Media",  
    founded: "1980",  
    location: "CA"  
  }  
}
```

Linking

In MongoDB documents can be related together as in the traditional relational database. It is possible to define one-to-one relations, one-to-many relations and many-to-many relations.

For example, let's suppose that for each unicorn we want to store its father (which is also stored in the same collection). Let's suppose that Leia is a puppy of Dunx (father) and Aurora (mother)

First thing to do: getting Dunx and Aurora ID

```
> db.unicorns.find({name: "Dunx"}, {"_id": 1});
```

```
{ "_id" : ObjectId("5cfe296e01c1e9419bca3e52") }
```

```
> db.unicorns.find({name: "Aurora"}, {"_id": 1});
```

```
{ "_id" : ObjectId("5cfe296e01c1e9419bca3e48") }
```

Linking

Let's modify Leia, adding the father:

```
> db.unicorns.update({name: "Leia"}, {$set: {parents:
ObjectId("5cfe296e01c1e9419bca3e52")}});
```

Let's make the same thing with Raleigh

```
> db.unicorns.update({name: "Raleigh"}, {$set: {parents:
ObjectId("5cfe296e01c1e9419bca3e52")}});
```

Now let's find all puppies of Dunx

```
> db.unicorns.find({parents: ObjectId("5cfe296e01c1e9419bca3e52")}, {name: 1, _id:
0})
```

Indexing

Search might be **time-consuming**

What is an index?

An index is a special data structure that store a small portion of the data set to make retrieves faster.

Without indexing, MongoDB **must** scan **every** document.

By default the only indexed field is the `_id`

To see all indexed field

```
> db.unicorns.getIndexes()
```

Indexing

Creating an index on a field

```
> db.unicorns.createIndex({name: 1})  
> db.unicorns.createIndex({name: -1})
```



1 = ascending ordering
-1 = descending ordering

Drop an existing index

```
> db.unicorns.dropIndex({name: 1})
```

Shows all indexes

```
> db.unicorns.getIndexes()
```

more indexes = larger spaces

Explaining

```
> db.zips.dropIndexes()
{
  "nIndexesWas" : 4,
  "msg" : "non-_id indexes dropped for collection",
  "ok" : 1
}
> db.zips.find({city: 'NASHVILLE', state: 'TN'}).explain()
{
  "cursor" : "BasicCursor",
  "isMultiKey" : false,
  "n" : 19,
  "nscannedObjects" : 29467,
  "nscanned" : 29467,
  "nscannedObjectsAllPlans" : 29467,
  "nscannedAllPlans" : 29467,
  "scanAndOrder" : false,
  "indexOnly" : false,
  "nYields" : 0,
  "nChunkSkips" : 0,
  "millis" : 33,
  "indexBounds" : {
  },
  "server" : "g:27017"
}
```

Without index

Returns a document that describes the process and indexes

```
> db.unicorns.find().explain()
```

```
> db.zips.find({city: 'NASHVILLE', state: 'TN'}).explain()
{
  "cursor" : "BtreeCursor state_1_city_1",
  "isMultiKey" : false,
  "n" : 19,
  "nscannedObjects" : 19,
  "nscanned" : 19,
  "nscannedObjectsAllPlans" : 19,
  "nscannedAllPlans" : 19,
  "scanAndOrder" : false,
  "indexOnly" : false,
  "nYields" : 0,
  "nChunkSkips" : 0,
  "millis" : 0,
  "indexBounds" : {
    "state" : [
      [
        "TN",
        "TN"
      ]
    ],
    "city" : [
      [
        "NASHVILLE",
        "NASHVILLE"
      ]
    ]
  },
  "server" : "g:27017"
}
```

With index

Aggregations

They are similar to Group By clause in all SQL systems.

In MongoDB, aggregations are realized using **pipe**, where the output of the former operator is sent as input to the next one.

The aggregate command takes an array of operators. The most important operator is **\$group**

Let's suppose we want to spot how many unicorns are male and how many are female:

```
> db.unicorns.aggregate([{$group: {_id: "$gender", total: {$sum: 1}}]);
```

The command creates a list of documents, each of them with two fields `_id` and `total`. The field `_id` corresponds to the gender while the `total` field is computed adding 1 every time the same gender is found

Aggregations

Beyond the \$group operator, there are many others

- \$match: similar to find, it aggregates documents satisfying the criteria
- \$sort: for sorting
- \$avg: compute the average

Example: we want to count the number of male and female unicorns having a weight greater than 600 kg and the mean number of vampires killed. The list must be ordered in descendent way depending on the mean number of killed vampires:

```
> db.unicorns.aggregate([{$match: {weight: {$gt: 600}}}, {$group: {_id: "$gender",  
total: {$sum:1}, meanVampires: {$avg:"$vampires"}}}, {$sort: {meanVampires: 1}} ]);
```

Aggregations

Another operator is \$unwind that flattens arrays

Which is the food loved the most by the unicorns?

```
> db.unicorns.aggregate([{$unwind: "$loves"}, {$group: {_id: "$loves", total: {$sum: 1}}}, {$sort: {total:-1}}, {$limit: 1}]);
```

Export e Import

Since MongoDB 4.4, this tool is released separately.

Download at:

<https://www.mongodb.com/download-center/database-tools/releases/archive>

wget:

https://fastdl.mongodb.org/tools/db/mongodb-database-tools-rhel80-x86_64-100.5.1.tgz

```
tar -xzvf mongodb-database-tools-rhel80-x86_64-100.5.1.tgz
```

```
mv mongodb-database-tools-rhel80-x86_64-100.5.1/bin/*  
/usr/local/mongodb-5.0.4/bin/
```

Export e Import

Export operations are usually done for backup

```
# mongodump --db learn --out myBackup
```

The directory myBackup is created. Inside, the learn database is stored. Data are stored in two files: bson (for data) e json (for metadata)

Restore:

```
# mongorestore --collection unicorns myBackup/learn/unicorns.bson
```

Export e Import

The mongoexport command allows to export data in json or csv format:

```
# mongoexport --db learn --collection unicorns > file.json
```

```
# mongoexport --db learn --collection unicorns --csv --fields name,weight,vampires
```